

ID Requisito	Descripción del Requisito	Elemento de Diseño (Clase/Interfaz)	Implementación (Método/Atributo clave)
RF-2.1.1.1	Gestión y modificación de packs por el Gestor.	Pack, Gestor	Pack.java: + addProducto(p: ProductoVenta, uds: int): void + eliminarLinea(p: ProductoVenta): void + modificarUnidades(p: ProductoVenta, cant: int): void + setPrecioOficial(precio: double): void + calcularPrecioFinal(): double Gestor.java: + crearDescuentoCantidad(...): void + crearDescuentoVolumen(...): void + crearDescuentoCategoria(...): void + crearDescuentoRegalo(...): void + eliminarDescuento(d: Descuento): void + modificarPrecioProducto(id: String, precio: double): void
RF-2.1.1.2	Consulta de estadísticas (Ingresos, Actividad, etc.).+1	Tienda, MotorEstadistico, Gestor	MotorEstadistico.java: + obtenerClientesConMasCompras(): List<Cliente> + obtenerClientesConMasIntercambios(): List<Cliente> + obtenerClientesConMasPedidosCaducados(): List<Cliente> + calcularIngresosRangoFechas(inicio, fin): double + calcularIngresosMesesAño(year): double[] + calcularIngresosMesesAñoActual(): double[] + calcularIngresosVenta(): double + calcularIngresosTasacion(): double Gestor.java: + verClientesTopCompras(): List<Cliente> + verClientesTopIntercambios(): List<Cliente> + verClientesConMasPedidosCancelados(): List<Cliente> + consultarIngresosRango(inicio, fin): double + consultarIngresosPorMeses(year): double[] + consultarIngresosVenta(): double + consultarIngresosTasacion(): double
RF-2.1.1.3	Organización y gestión de categorías.	Categoria, Gestor	Gestor.java: + crearCategoria(nombre: String): void + añadirProductoACategoria(p: ProductoVenta, cat: Categoria): void + eliminarProductoDeCategoria(p: ProductoVenta, cat: Categoria): void Categoria.java: setters y getters

RF-2.1.1.4	Descuentos: Creación, aplicación y tipos (Cantidad, Volumen, Categoría, Regalo).+4	Descuento e hijos (DescuentoCantidad, DescuentoCategoría, DescuentoVolumen, Regalo)	En cada subclase: + calcularDescuento(carrito: Carrito): double Gestor.java: + crearDescuentoCantidad(desde: int, hasta: int, descuento: double, prioridad: int, inicio: LocalDate, fin: LocalDate): void + crearDescuentoVolumen(minUds: int, descuento: double, prioridad: int, inicio, fin): void + crearDescuentoCategoría(cat: Categoría, descuento: double, prioridad: int, inicio, fin): void + crearDescuentoRegalo(precioMin: double, regalo: ProductoVenta, prioridad: int, inicio, fin): void + eliminarDescuento(d: Descuento): void
RF-2.1.1.4	No acumulabilidad: Prevalece el primer descuento impuesto por el gestor.+2	Carrito, Tienda, Descuento	Carrito.java: + getDescuentoAplicado(): Descuento + setDescuentoAplicado(d: Descuento): void Descuento.java: atributo ordenPrioridad + getter Tienda.java: + aplicarDescuentoPrioritario(carrito: Carrito): void + limpiarDescuentosCaducados(): void
RF-2.1.1.5	Gestión de permisos de empleados.+1	Gestor, Empleado	Empleado.java: + tienePermiso(p: TipoPermisos): boolean + asignarPermiso(p: TipoPermisos): void + quitarPermiso(p: TipoPermisos): void + getPermisos(): List<TipoPermisos> Gestor.java: + asignarPermiso(e: Empleado, p: TipoPermisos): void + retirarPermiso(e: Empleado, p: TipoPermisos): void
RF-2.1.1.6	Gestión de precios de productos.	Producto, ProductoVenta, Gestor	ProductoVenta.java: + getPrecioOficial(): double + setPrecioOficial(precio: double): void Gestor.java: + modificarPrecioProducto(id: String, nuevoPrecio: double): void
RF-2.1.1.7	Configuración de parámetros del sistema de recomendación (pesos).	Recomendador, Gestor	Recomendador.java: + setPesos(pesoValoracion: double, pesoCompras: double, pesoCategorias: double): void + setConfiguracion(activo: boolean, limiteMax: int): void + getPesoValoracion(): double + getPesoCompras(): double + getPesoCategorias(): double + isActive(): boolean Gestor.java: (accede al Recomendador vía Tienda.getRecomendador())

RF-2.1.1.8	Alta/Baja de empleados y gestión de credenciales (inmutables).	Gestor, Empleado	Gestor.java: + darDeAltaEmpleados(nick: String, pass: String, email: String, permisos: List<TipoPermisos>): Empleado + darDeAltaEmpleados_Permisos(nick: String, pass: String, email: String): Empleado + darDeBajaAEmpleado(e: Empleado): void Empleado.java: credenciales (nickname/password) son inmutables desde fuera (no hay setter público)
RF-2.1.1.10	Tiempo máx. bloqueo productos intercambio.	Tienda, Oferta	Tienda.java: + getTiempoMaxOferta(): long + setTiempoMaxOferta(t: long): void (solo vía Gestor) Oferta.java: + haCaducado(): boolean + getEstado(): EstadoOferta + setEstado(e: EstadoOferta): void ComprobadorTiempo.java: revisarCarritosCaducados() gestiona expiración periódica
RF-2.1.1.11	Tiempo máx. bloqueo productos en carrito.	Tienda, Carrito, GestorTiempo	Tienda.java: + getTiempoMaxCarrito(): long + setTiempoMaxCarrito(t: long): void (solo vía Gestor) Carrito.java: + estaCaducado(): boolean + caducar(): void ComprobadorTiempo.java: + revisarCarritosCaducados(): void + iniciarRevisionPeriodica(): void
RF-2.1.1.12	Tiempo máx. reserva de pedido pendiente de pago.+1	Tienda, Pedido, GestorTiempo	Tienda.java: + getTiempoMaxPago(): long + setTiempoMaxPago(t: long): void (solo vía Gestor) Pedido.java: + isCaducado(): boolean + cancelarPedido(): void + actualizarEstado(est: EstadoPedido): void ComprpbadorTiempo.java: + revisarPedidosPendientesCaducados(): void + cancelarPedidoPendiente(idPedido: String): void

RF-2.1.2.1	Gestión de stock (Manual y por fichero de texto).+1	Empleado, Tienda	Empleado.java: + añadirProducto_nuevo(p: ProductoVenta): void + reponerStockProducto(p: ProductoVenta, uds: int): void + cargarProductosFicheroTexto(path: String): void + procesarNuevoProducto(linea: String): ProductoVenta + actualizarStockDesdeLinea(linea: String): void Tienda.java: + añadirProducto(p: ProductoVenta): void + getStockVentas(): List<ProductoVenta>
RF-2.1.2.2	Gestión de packs por empleados (crear, modificar, eliminar).	Empleado, Pack	Pack.java: + addProducto(p: ProductoVenta, uds: int): void + eliminarLinea(p: ProductoVenta): void + modificarUnidades(p: ProductoVenta, cant: int): void + setPrecioOficial(precio: double): void + calcularPrecioFinal(): double Empleado.java: + crearPack(productos: List<ProductoVenta>, descuento: double): Pack + añadirProductoaPack(pack: Pack, p: ProductoVenta, uds: int): void
RF-2.1.2.4	Modificación de información de productos (descripción, imagen).	Empleado, ProductoVenta	ProductoVenta.java: + setDescripcion(desc: String): void (heredado de Producto) + setImagenRuta(ruta: String): void (heredado de Producto) Empleado.java: + modificarDescripcionProducto(p: ProductoVenta, desc: String): void + modificarImagenProducto(p: ProductoVenta, ruta: String): void
RF-2.1.2.5	Gestión de pedidos y estados de seguimiento.+1	Empleado, Pedido	Pedido.java: + setEstado(est: EstadoPedido): void + actualizarEstado(est: EstadoPedido): void + marcarPreparado(): void + marcarEntregado(): void + getEstado(): EstadoPedido Empleado.java: + prepararPedido(p: Pedido): void + entregarPedido(p: Pedido): void + buscarPedidoPorId(id: String): Pedido

RF-2.1.2.6	Tasación y valoración de productos (estados de conservación).+1	Empleado, Producto2Mano, Valoracion, Cliente	Producto2Mano.java: + getValoracion(): Valoracion + setValoracion(v: Valoracion): void + isBloqueado(): boolean Valoracion.java: EstadoProducto, EstadoValoracion (enums) + pagar(tarjeta: String, cvv: int, caducidad: Date): boolean + getEstadoProducto(): EstadoProducto + getEstadoValoracion(): EstadoValoracion Cliente.java: + subirProducto(p: Producto2Mano): void + solicitarTasacion(p: Producto2Mano, tarjeta: String, cvv: int, caducidad: Date): void Empleado.java: + tasarProducto(p: Producto2Mano, precio: double, estado: EstadoProducto): void + buscarProductoPendientePorId(id: String): Producto2Mano Tienda.java: + getPendientesTasacion(): List<Producto2Mano>
RF-2.1.2.7	Confirmación de intercambios acordados entre clientes por empleados autorizados.	Empleado, Oferta, Tienda	Empleado.java: + confirmarIntercambio(o: Oferta): void [requiere permiso CONFIRMACION_INTERCAMBIO] + puedeRealizarTarea(p: TipoPermisos): boolean Oferta.java: + aceptarYEjecutar(): void + setEstado(e: EstadoOferta): void + getEstado(): EstadoOferta Tienda.java: + getCatalogoIntercambio(): List<Producto2Mano> + registrarIntercambioFinalizado(o: Oferta): void
RF-2.1.2.8	Alta de empleados por el gestor. Credenciales inmutables para el empleado.	Gestor, Empleado	Gestor.java: + darDeAltaEmpleados(nick: String, pass: String, email: String, permisos: List<TipoPermisos>): Empleado + darDeAltaEmpleados_Permisos(nick, pass, email): Empleado + darDeBajaAEmpleado(e: Empleado): void Empleado.java: no tiene método de cambio de credenciales

RF-2.1.2.9	Entrega física de pedidos en tienda y actualización de estado a "Entregado".	Empleado, Pedido	Empleado.java: + entregarPedido(p: Pedido): void Pedido.java: + marcarEntregado(): void + setEstado(EstadoPedido.ENTREGADO): void
RF-2.1.3.1	Búsqueda de productos nuevos en el catálogo.	Cliente, UsuarioRegistrado, Tienda, FiltroVenta	UsuarioRegistrado.java: + buscarProductos(): List<ProductoVenta> + buscarProductosPorNombre(nombre: String): List<ProductoVenta> + buscarProductoPorId(id: String): ProductoVenta + buscarProductosPorCategoria(cat: Categoria): List<ProductoVenta> + buscarProductosVentaFiltrados(f: FiltroVenta): List<ProductoVenta> + filtrarPorPrecio(min: double, max: double): void + filtrarPorCategoria(cat: Categoria): void + filtrarPorPuntuacion(min: double): void + filtrarProductos(): List<ProductoVenta> FiltroVenta.java: + productoCumpleFiltro(p: ProductoVenta): boolean + resetear(): void + setPrecioMinimo/Maximo/PuntuacionMinima/añadirCategoria(): void Tienda.java: + buscarProductosFiltrados(f: FiltroVenta): List<ProductoVenta> + buscarProductoVenta(): List<ProductoVenta> + buscarProductoVentaPorId(id: String): ProductoVenta + buscarproductoPorNombre(nom: String): List<ProductoVenta>
RF-2.1.3.2	Búsqueda de productos de intercambio (2ª mano).	Cliente, UsuarioRegistrado, Tienda, FiltroSegundaMano	UsuarioRegistrado.java: + buscarProductosSegundaMano(): List<Producto2Mano> + buscarProducto2ManoNombre(nom: String): List<Producto2Mano> + buscarProducto2ManoPorId(id: String): Producto2Mano + buscarProductos2ManoFiltrados(f: FiltroSegundaMano): List<Producto2Mano> + filtrar2ManoPorValor(min: double, max: double): void + filtrar2ManoPorEstado(estado: EstadoProducto): void + filtrar2Mano(): List<Producto2Mano> FiltroSegundaMano.java: + cumpleFiltro(p: Producto2Mano): boolean + resetear(): void + setValorMinimo/Maximo/EstadoMinimo(): void Tienda.java: + buscarSegundaMano(): List<Producto2Mano> + buscarSegundaManoPorNombre(nom: String): List<Producto2Mano> + buscarSegundaManoFiltrado(f: FiltroSegundaMano): List<Producto2Mano>

RF-2.1.3.3	Compra de productos (Exclusivo registrados).	Cliente, Carrito, Pedido, GestorTiempo, Tienda	Cliente.java: + añadirProductoCarrito(pV: ProductoVenta): void + reservarCarrito(carrito: Carrito): Pedido + pagarCarrito(idPedido: String, tarjeta, cvv, caducidad): boolean + solicitarRecogidaPedido(idPedido: String): void Carrito.java: + añadirProducto(pV: ProductoVenta): void + eliminarProducto(pV: ProductoVenta): void + cambiarCantidadProducto(pV: ProductoVenta, cant: int): void + getTotal(): double Pedido.java: + pagar(tarjeta: String, cvv: int, caducidad: Date): boolean ComprobadorTiempo.java: + crearPedidoDesdeCarrito(carrito: Carrito): Pedido + pagarPedidoPendiente(idPedido, tarjeta, cvv, caducidad): boolean Tienda.java: + aplicarDescuentoPrioritario(carrito: Carrito): void
RF-2.1.3.4	Reseñas y puntuación de productos comprados.	Cliente, Reseña, ProductoVenta	Cliente.java: + escribirReseña(p: ProductoVenta, nota: double, comentario: String): void + productoHasidoPedidoYentregado(p: ProductoVenta): boolean ProductoVenta.java: + addReseña(r: Reseña): void + deleteReseña(r: Reseña): void + getReseñas(): List<Reseña> + getMediaPuntuacion(): double UsuarioRegistrado.java: + verReseñasProducto(p: ProductoVenta): List<Reseña>

RF-2.1.3.5	Subir productos a cartera personal previa tasa de tasación.	Cliente, Producto2Mano, Valoracion	Cliente.java: + subirProducto(p: Producto2Mano): void + solicitarTasacion(p: Producto2Mano, tarjeta: String, cvv: int, caducidad: Date): void + getCarteraIntercambio(): List<Producto2Mano> Valoracion.java: + pagar(tarjeta: String, cvv: int, caducidad: Date): boolean Tienda.java: + solicitarTasacion(p: Producto2Mano, tarjeta, cvv, caducidad): void + getPendientesTasacion(): List<Producto2Mano>
RF-2.1.3.6	Acceso a carteras de otros clientes para ver ofertas.	Cliente, UsuarioRegistrado	UsuarioRegistrado.java: + verCarteraCliente(c: Cliente): List<Producto2Mano> Cliente.java: + getCarteraIntercambio(): List<Producto2Mano> + crearListaProductos2Mano(): List<Producto2Mano>
RF-2.1.3.7	Acceso al historial de intercambios realizados.	Cliente, Oferta	Cliente.java: + verMiHistorialIntercambios(): List<Oferta> + getHistorialIntercambios(): List<Oferta> + verIntercambioscon(c: Cliente): List<Oferta> + verMisOfertasEnviadas(): List<Oferta> + verMisOfertasPorResponder(): List<Oferta> Tienda.java: + getIntercambiosFinalizados(): List<Oferta> + registrarIntercambioFinalizado(o: Oferta): void

RF-2.1.3.8	Notificaciones: Descuentos, pagos con éxito, caducidad de pedidos.	Notificacion, PreferenciaNotificacion, Cliente, Tienda	Notificacion.java: + marcarComoLeida(): void + getMensaje(): String + getTipo(): TipoNotificacion Cliente.java: + recibirNotificacionTipo(tipo: TipoNotificacion, msg: String): void + configurarPreferenciaNotificacion(tipo: TipoNotificacion, activo: boolean): void + añadirCategoriaInteresParaRecibirInfo(cat: Categoria): void + getNotificacionesNoLeidas(): List<Notificacion> + verNotificacion(n: Notificacion): void + verMisNotificaciones(): void Tienda.java: + registrarNotificacion(dest: UsuarioRegistrado, msg: String, tipo: TipoNotificacion): void + notificarDescuento(d: Descuento): void + getNotificacionesNoLeidas(u: UsuarioRegistrado): List<Notificacion> Pedido.java: al pagar() envía código de recogida vía notificación Carrito.java: al caducar() se notifica al cliente
RF-2.1.4.1	Cliente no registrado: Solo búsqueda de productos tienda.	UsuarioNoRegistrado	UsuarioNoRegistrado.java: + buscarProductos(): List<ProductoVenta> + buscarProductosPorNombre(nom: String): List<ProductoVenta> + buscarProductoPorId(id: String): ProductoVenta + buscarProductosPorCategoria(cat: Categoria): List<ProductoVenta> + registrarse(nick, pass, email, dni): Cliente
RF-2.1.4.2	Cliente no registrado: No accede a carteras ni sistema de intercambio.	UsuarioNoRegistrado	UsuarioNoRegistrado.java: NO tiene métodos de intercambio, cartera, ni compra. Solo acceso a búsqueda de catálogo de venta. (no hay proponerOferta, subirProducto, etc.)